

TiDAR Feature Card: Coefficient-Gated Loss Branch Pruning in JAX

What this feature is

In TiDAR training, each auxiliary loss term is optional and controlled by a coefficient:

- alpha: AR CE
- beta: Diffusion CE
- rho: forward KL
- chi: reverse KL
- delta: hard agreement
- eta: distillation KL
- gamma: top-k set distillation

The key implementation detail is:

- if a coefficient is 0.0, the corresponding branch is excluded from that compiled executable,
- if a coefficient is non-zero, that branch is included.

This gives a clean way to run many training variants without maintaining separate codepaths.

Why it matters

- It reduces unnecessary compute when ablation terms are off.
- It keeps one source of truth for all loss variants.
- It enables stage-wise objectives by changing only config values.
- It improves experiment velocity for a diploma workflow (fast toggles, same script).

Where it is implemented

Python-side branch gating in TiDAR/model/Training_step.py:

```
1  if alpha > 0.0:
2      ... # AR CE
3
4  if beta > 0.0:
5      ... # Diffusion CE
6
7  if (rho > 0.0) or (chi > 0.0) or (delta > 0.0) or (eta > 0.0):
8      ... # Alignment family
9
10 if (gamma > 0.0) and (gamma_topk > 0):
11     ... # Top-k set distillation
```

Static arguments in TiDAR/model/Run_training.py inside jitted _run_chunk:

```
1  @partial(
2      jax.jit,
3      static_argnames=(
4          "alpha", "beta", "rho", "chi",
5          "delta", "eta", "eta_T", "gamma", "gamma_topk",
6      ),
7  )
8  def _run_chunk(...):
9      ...
```

Because coefficients are static, each unique tuple of values gets its own compiled variant.

Verification summary (compile behavior)

Probe script run with:

- JAX_LOG_COMPILES=1
- JAX_EXPLAIN_CACHE_MISSES=1

Observed behavior:

- First call with gamma=0.0 compiled once.
- Second call with gamma=0.0 reused cache.
- Switching to gamma=0.1 caused a new compile (static-arg cache miss).
- Second call with gamma=0.1 reused cache.

Raw log excerpt from the probe run:

```

1 $ JAX_LOG_COMPILES=1 JAX_EXPLAIN_CACHE_MISSES=1 /Users/antonhrstov/v/SG/bin/python /
  tmp/tidar_compile_probe.py
2 ...
3 WARNING:jax._src.pjit: TRACING CACHE MISS ...
4   never seen function: compiled_loss_and_grad
5 WARNING:jax._src.interpreters.pxla: Compiling jit(compiled_loss_and_grad) ...
6 ...
7 WARNING:jax._src.pjit: TRACING CACHE MISS ...
8   all previously seen cache keys are different.
9   key with different value of static kwargs:
10  now {... gamma: 0.1, gamma_topk: 8, ...}
11  before {... gamma: 0.0, gamma_topk: 8, ...}
12 WARNING:jax._src.interpreters.pxla: Compiling jit(compiled_loss_and_grad) ...

```

Interpretation:

- The first miss is expected for first-ever trace/compile.
- The second miss is specifically due to static argument change (gamma), proving a new compiled variant.
- No additional compile lines appear for the repeated same-coefficient calls in that run.

JAXPR check also confirmed pruning for the top-k branch:

- top_k primitive count with gamma=0.0: 0
- top_k primitive count with gamma=0.1: 1

Practical implication for experiments

For TiDAR ablations, setting a coefficient to zero is not only a semantic “off” switch. It also creates a lighter compiled graph for that run configuration.

This is useful for staged training, where early stages can focus on core AR+Diff losses, and later stages can enable selected agreement terms only when needed.

Config examples

```

1 # exmaple config with 3 losses
2 training:
3   loss:
4     alpha: 1.0
5     beta: 1.0
6     rho: 0.0
7     delta: 0.5
8     gamma: 0.0
9     gamma_topk: 4

```

gamma_topk should be paired with gamma.

- If gamma = 0.0, top-k set loss is disabled regardless, so gamma_topk is effectively unused.

- If top-k set loss is enabled, use a non-zero pair like: